Adaptive Reflex Engine: Partner Brief

2026-09-21 · @Someone

What it is

The Adaptive Reflex Engine is a deterministic layer that sits in front of a language model and answers quantitative questions in one of three ways: it certifies an answer it computed itself, it passes the question to the model and labels the reply as the model's, or it abstains. The one guarantee it offers is that a certified answer is right: across 23 pre-registered experiments and several million evaluations on sealed data, the certified path has never served a wrong answer at scale. Everything the engine cannot certify is handed on with no claim attached, so a caller always knows which answers carry the guarantee and which carry the model's ordinary accuracy.

What we achieved

The numbers below come from sealed test sets that were hashed before any run and scored once. Nothing was tuned on them.

Result	Measurement	Where
Certified answers are right	0 wrong in 133,450 certified serves across four Claude models, plus 29,941 with no model at all	V1.10, 2026, 92,392-item sealed set
The property holds across versions	0 wrong in 118,000 (V1.6), 0 in 112,170 (V1.7), 0 binder-attributed wrongs in 11.4 million evaluations (V1.8)	2025 to 2026
Share of mixed quantitative traffic certified without a model call	35 to 38 percent of a 387,062-item workload of word problems, physics chains and science questions	V1.10
Concept questions that must never be certified	0 certified on every model arm; 100 percent escalated	V1.10, SciQ items

Result	Measurement	Where
Cost of a certified answer	0.110 J and about 1 millisecond of compute per serve, metered with positive controls; zero model tokens	V1.8
Works with four model sizes	Haiku, Sonnet, Opus, Fable; more capable models produce fewer wrong plans and more coverage	V1.6 to V1.10

One exception is on record: on the second sealed sample of V1.10, one stored physics plan computed weight where force was asked, because the plan's index key did not carry the unit of the question. It is the only certified wrong in the arc's sealed history, it is fixable with a unit check on the plan before serving, and it is listed rather than hidden.

Where it is useful to Accelerated Understanding

Accelerated Understanding builds neural-operator models that simulate physical systems for discovery, chip design, robotics and weather. Those models produce numbers that people will act on, and the layer around them (the questions users ask, the derived quantities, the unit handling, the assistant that explains a result) is where a confidently wrong number does the damage. The engine was built for exactly that layer.

- **A certified arithmetic and physics layer around simulation outputs.** Derived quantities, unit conversions, chained formula evaluations and comparisons against stated numbers are the engine's certified path: 0 wrong across 133,450 sealed serves, with dimensional gates on every plan. A scientific assistant can hand these to the engine and reserve the model for what only the model can do.
- **An honest abstention.** The engine refuses to certify concept questions and ambiguous asks (0 concept items certified on any arm in the latest run). For a scientific product, an explicit "not computed" beats a fluent guess, and the engine makes that distinction visible to the user.
- **Auditable certificates.** Every certified value names the plan and the bound inputs, so a result in a discovery workflow can be replayed. Model outputs are returned labelled as the model's, never edited or filtered, because we measured that filtering by agreement or self-check does not remove a model's genuine errors.
- **Evaluation discipline for a physics-AI company.** Pre-registered thresholds, hashed sealed sets scored once, positive controls on every instrument, a numbered defect ledger and post-hoc error classes published beside raw counts. This protocol is what let us find and report our own failures (13 of 23 experiments graded negative or partial), and it applies directly to validating a simulation model's claims.

- **Cost on the quantitative share.** Certified answers cost about a millisecond of CPU and zero model tokens. On repetitive templated traffic the certified share reached 95 percent; on mixed real questions it is about a third.

We have not evaluated the engine on PDE-scale simulation or on neural-operator outputs; our physics work is single-formula and chained textbook physics with roles and dimensions. What we bring is the certified front layer and the measurement protocol, and we would want the pilot to test how far the certified path extends into the quantities your models actually produce.

How it fits in front of your model

Integration is a single call. The engine takes the question text, returns a verdict and, for certified answers, a certificate.

```
flowchart LR
  Q[Question] --> E[Reflex Engine]
  E -->|structure matched and verified| C[CERTIFIED answer + certificate]
  E -->|no verified structure| M[Your model]
  M --> L[ESCALATED answer, labelled]
  E -->|ask is a concept or ambiguous| A[ABSTAIN]
```

- **CERTIFIED**: the engine matched the question to a plan it minted and verified, bound the stated numbers, and computed the result itself. Zero model tokens. Comes with the plan and the bindings.
- **ESCALATED**: the question goes to whichever model you already run. The reply is returned unchanged with the label. The engine never edits or filters a model answer, because we measured that filtering by agreement, self-check or corroboration does not remove the model's genuine errors.
- **ABSTAIN**: concept questions, ambiguous asks and questions with no usable numbers are declined explicitly.

The engine learns new plans from traffic offline, with a model proposing structure and the engine verifying and gating every proposal before it can be reused. Learning costs model tokens; serving does not. It runs on a CPU, holds its library in a JSON file, and adds about a millisecond per question.

What it does not do

- It does not make your model more accurate. Escalated answers carry the model's own accuracy, and we have tested and rejected every cheap way of filtering them (corroboration, self-checks, verifiers, library agreement, key consensus).

- It does not certify most of a mixed workload. On real word problems the certified share is about a third; on long multi-step stories and physics word problems it is near zero today. On repetitive templated traffic it is far higher.
- It does not reduce total tokens when the model is asked to propose structure for every question at serve time. We measured that pipeline at two to three times a plain model call. The engine saves tokens only on what it certifies, which is why serving and learning are separated.
- It has been evaluated on English quantitative questions (arithmetic word problems, physics, science concepts) with Claude models and one local 8B model. Other languages, domains and models are untested.
- It is a research codebase with a reproducible evaluation harness, not a packaged product. Deployment work would be part of a pilot.

How we work

The evaluation discipline is as much the asset as the engine, and it transfers to any system that puts a model in front of users.

- Every version's hypotheses, thresholds and outcome grade are frozen in a signed tag before the run; the grade is whatever the frozen ladder says.
- Test sets are hashed before any run and scored once. A second sealed sample checks that results reproduce.
- Every measuring instrument has a positive control that must fail when it should; this rule caught an energy meter reading zero and a token count the model had invented.
- Every defect is numbered, its run preserved, and a superseding run added. Forty-three are on record. Nothing is deleted or relabelled.
- Wrong answers are classified after the run (genuine, corpus error, ambiguous) and the classification is published beside the raw count, never instead of it.
- Models are reached only through sandboxed agents that see questions and nothing else: no answers, no verifier, no engine code.

Thirteen of the 23 experiments returned a negative or partial grade, and each is reported as prominently as the positive ones. A partner gets the same treatment: the pilot's success criteria are written and frozen before it runs, and the report says what happened.

What we propose

A six-week pilot on a sample of your own traffic, run under the same protocol as our experiments.

1. **Week 1.** You provide 5,000 to 20,000 historical quantitative questions with their accepted answers, under NDA. We freeze the success criteria with you: the certified share we expect, the wrong-certified bar (zero), and the token saving on the certified share.
2. **Weeks 2 to 4.** The engine acquires plans from a development split of your traffic, using your model or ours for structure proposals. No tuning touches the sealed split.
3. **Week 5.** One sealed run. We report certified share, certified wrongs by class, escalation rate, abstentions and tokens per query against your baseline.
4. **Week 6.** A written report with the frozen criteria, the numbers, every defect found, and a recommendation: integrate, extend acquisition, or stop.

What we need from you: the traffic sample, a contact who knows what an acceptable answer looks like in your domain, and read access to your model endpoint or a token budget for ours.

What you get regardless of outcome: an honest measurement of how much of your quantitative traffic can be served with a guarantee, and a sealed-evaluation harness you can keep using on your own systems.

Contact: Kavan Shah, kavanshah112@gmail.com.